SOKOLINK

# SokoLink — Production Plan (Python)

**Status:** draft, 2026-08-05. Supersedes the 14-day TypeScript plan. Mirrored into Notion once the connector is re-authorized.

## The product

```
SokoLink
│
├── Soko Commerce          selling
│   ├── Catalog
│   ├── Orders
│   ├── Payments
│   └── WhatsApp Storefront
│
├── Soko Intel             getting seen
│   ├── Scripts
│   ├── Captions
│   ├── Market Insights
│   └── Competitor Intelligence
│
└── Soko AI                the conversation
    ├── Customer Support
    ├── Sales Agent
    ├── Follow-ups
    └── Order Assistance
```

*SokoLedger is parked — Phase 2, not in this plan.*

## The decision

**Build SokoLink fresh, in Python. `Project TIKTOK` is a reference, not a foundation.**

TIKTOK proved the hard mechanics — a vision model reads a price off a TikTok cover, Gemini hears one spoken in Sheng, Daraja callbacks can be made idempotent. That knowledge carries. The code does not get adopted wholesale, because production SokoLink is a different product.

**Language: Python — FastAPI + SQLAlchemy + Alembic + Pydantic.** Chosen because Fredrick is materially stronger in Python and can debug it unaided; on a solo build the maintainer's fluency beats framework elegance. Pydantic does double duty as API-boundary validation *and* LLM structured-output schema.

### TIKTOK modules worth reading before writing the equivalent

| MODULE | WHAT IT ALREADY SOLVED |
|---|---|
| `services/scraper.py` | Apify behind our own adapter, so the engine swaps in one file |
| `agent/draft.py` | The price cascade: cover image, then Gemini watches **and listens**. Live at KES 500/550/550/600 |
| `agent/sales.py` | Grounded Sheng-aware chat via context injection, not RAG |
| `services/mpesa.py` · `api/daraja.py` | Daraja OAuth, STK push, **idempotent** callback |
| `models/` + Alembic | DB rails: stock ≥ 0, unique video id, publish-needs-price |

Also read in full: TIKTOK's `docs/CONCEPTS.md` and `docs/WORKPLAN.md`.

---

## The buyer journey

```
WhatsApp chat
      ↓
Browse products
      ↓
WhatsApp in-app browser opens
      ↓
SokoLink storefront          ← mobile-first web, server-rendered
      ↓
Product selection
      ↓
M-Pesa checkout
      ↓
Return to WhatsApp
```

Rather than pushing buyers to a separate website, SokoLink launches a mobile-first web experience **from inside WhatsApp**.

### Why this is the right call

- **No dependency on WhatsApp Flows approval.** The in-app browser is an ordinary webview; it works the day the Cloud API is live.
- **Full control of the interface.** No Meta component vocabulary to design around.
- **Debuggable in a normal browser** — which is half the reason we chose Python.
- **One surface, two jobs.** The same server-rendered pages are the in-app storefront *and* the publicly indexable web presence. No second app to maintain.

### The three hard parts (design these deliberately)

**1. Identity across the hop.** The storefront must know which buyer and which shop opened it, without the buyer logging in. The link carries a **signed, short-lived token** minted when the chat sends it.

> Never put a raw phone number in the URL. URLs leak — through history, referrer headers, and buyers forwarding links to friends. A forwarded link must not carry someone else's identity. Sign it, scope it, expire it.

**2. M-Pesa interrupts the browser.** The STK prompt takes over the same phone the webview is on. When the buyer returns, the page must already know what happened — so checkout **polls order status** and renders paid / failed / timed-out states honestly. The Daraja callback remains the only payment truth; the page merely reflects it.

**3. Getting back to WhatsApp.** The return hop is part of the design, not an afterthought — a confirmed order should land the buyer back in the conversation with the seller, not stranded on a success page.

---

## Rules carried forward (non-negotiable)

- **The agent proposes, code disposes.** The LLM never decides price, stock, payment status, or consent.
- **Publish is a human gate**, and still requires a price.
- **The payment callback is the only payment truth**, processed idempotently — Daraja retries, and so does Meta.
- **No RAG** unless a genuine similarity problem appears (see the hook corpus).
- **No AI framework.** Provider SDKs directly, Pydantic as the guardrail.
- **One database per application.** Learned in TIKTOK 0.2; nearly re-learned on 2026-08-05 when a shared Railway DB almost got reset.
- **Rails before agent.** Payment paths are built and tested before an agent may call them.
- **Spike before schema.** Let real payloads drive the data model.
- **Every file opens with a header comment**; comments explain *why*. Tests ship with the logic.

---

# Phase 1 — Soko Commerce

*Nothing else matters until a seller can sell.*

### P0 — Foundation `p0-foundation`

- FastAPI + SQLAlchemy + Alembic + Pydantic; **its own Railway Postgres**.
- Typed config via `pydantic-settings`; secrets only in `.env`.
- `/health` with liveness and DB readiness.
- pytest with transactional fixtures; CI green on every PR.
- Retire the TypeScript scaffold.

**Done when:** the app boots against its own database and CI is green.

---

### P1 — Catalog `p1-catalog`

*Goal: products reach the catalogue by whichever route suits the seller.*

**Three ingestion paths, one pipeline.** Real sellers are not all alike: some have a feed worth bulk-importing, some want to add one specific video, and some have stock that was never filmed. A single front door fails two of those three.

| PATH | SELLER DOES | SYSTEM DOES |
|---|---|---|
| **Profile sync** | enters `@handle` once | Apify pulls the feed → AI drafts each video |
| **Single link** | pastes one TikTok URL | same pipeline, one item |
| **Manual upload** | uploads a photo | no scrape; AI drafts from the uploaded image |

The third path reuses the vision agent unchanged — it does not care whether the image came from a TikTok cover or the seller's camera roll. Same code, new door.

- **Spike first**: re-validate the Apify actor and payload shape.
- `services/scraper.py` — Apify behind our own interface. Thumbnails stored by us; TikTok CDN URLs expire.
- Seller and Product models with DB-level rails.
- **The price cascade**: caption hints → cover image → Gemini watches and listens. Structured output against a Pydantic schema; the model drafts, publish is human.
- Each video processed **once ever**, keyed by video id, cached.

**Data-model consequences of the three paths:**

- `Product.source` — `tiktok_profile` / `tiktok_link` / `manual`. Provenance decides what may be re-synced and what must never be overwritten.
- `tiktok_video_id` is **nullable** with its unique constraint intact — Postgres permits multiple NULLs in a unique index, so manual products coexist cleanly.
- **A profile re-sync must never touch manually-created products.** Enforced as a rail with its own test, not left to convention: a seller who adds stock by hand, syncs their feed, and watches it vanish does not come back.
- The full cascade applies to paths 1 and 2. Manual uploads reach tier 2 (cover image) only — there is no video to listen to.

**Done when:** all three paths produce reviewable drafts, and a profile re-sync leaves manual products untouched.

---

### P2 — WhatsApp channel `p2-wa-channel`

- Cloud API webhook: signature verification, `hub.challenge` handshake, receive.
- Send: text, media, interactive messages, and the storefront link.
- **Idempotency at the channel edge** — Meta redelivers; a replay must never double-charge or double-reply.
- Conversation session state keyed by phone number.
- **Signed storefront links** — mint the scoped, expiring token described above.
- Structured logging, one correlation id per conversation.

**Done when:** a real WhatsApp message gets a correct reply, and a replayed webhook changes nothing.

---

### P3 — Storefront + Seller Dashboard `p3-storefront`

*Two surfaces, two audiences, one stack: Jinja2 + HTMX + Tailwind.*

Chosen so the whole application stays one language and one deploy — the reason Python was picked in the first place. HTMX covers inline editing, bulk actions, upload progress and live filtering without a JavaScript framework or a second debugging context.

**Buyer storefront** — opened in the WhatsApp in-app browser:

- Server-rendered, mobile-first. Minimal client JS — buyers are on cheap Android handsets over expensive data.
- Token-scoped: the page knows the shop and the buyer without a login.
- Catalogue → product detail → selection.
- Designed empty, error and sold-out states. A stale price must be impossible.
- **Publicly indexable shop pages** with correct OG/Twitter metadata, so a link pasted into TikTok or WhatsApp previews properly — this is also the SEO surface.

**Seller dashboard** — the working surface, often on a laptop:

- All three ingestion paths: sync a handle, paste a link, upload a photo.
- **Draft review queue** — the primary screen. Bulk-confirm what the AI got right, correct what it did not. Low-confidence parses surface first, because those are where a wrong price hides.
- Inline edit of price, title, sizes and stock, without a page reload.
- Publish, with the publish-requires-price gate visible rather than mysterious — a disabled button that says *why* it is disabled.
- At-a-glance state: how many drafts wait, what is live, what is sold out.
- Sync status and last-synced time, so "why has nothing changed?" has an answer on the page instead of in a support message.

**Done when:** a buyer completes a selection from WhatsApp, and a seller takes a product from any of the three paths to published without leaving the dashboard.

---

### P4 — Orders + Payments `p4-payments`

- Order state machine; Kenyan phone normalisation; consent as explicit data.
- Daraja client: OAuth, STK push, status query.
- **Idempotent callback** — callback is truth, stock decrements exactly once.
- Checkout page polls order status and survives the STK interruption.
- Return-to-WhatsApp on completion.

**Done when:** a buyer completes a sandbox M-Pesa payment from the storefront and lands back in the chat with the order confirmed.

---

# Phase 2 — Soko AI

*The conversation layer, on top of a working shop.*

### P5 — Sales Agent + Customer Support `p5-soko-ai`

- Agent grounded **only** in the shop's published catalogue — context injection, not RAG. Sheng-aware.
- Natural buyer questions answered in chat: sizes, colours, availability, price.
- Honest about sold-out; never invents a variant that does not exist.
- Handoff to the seller when the agent cannot answer.
- Tools ( `check_stock` , `create_order` , `send_stk_push` ) only after P4's rails exist and are tested.

**Done when:** a buyer asks real questions in WhatsApp, gets grounded answers, and is guided to a completed purchase.

---

### P6 — Follow-ups + Order Assistance `p6-followups`

- Order status on request — "where is my order?" answered from real state.
- Post-purchase follow-up; delivery and support conversation.
- Re-engagement and restock notifications: **opt-in only, STOP honoured, paced**. The agent drafts, the seller approves, code enforces the caps.

**Done when:** a buyer can ask about an order and get a truthful answer, and a seller can send an approved restock nudge without touching the send button twice.

---

# Phase 3 — Soko Intel

*Getting the seller seen. Independent of Phases 1–2 — can run in parallel once Commerce is stable.*

### P7 — Competitor Intelligence + Market Insights `p7-intel-insights`

- **Spike first**: pin down which Apify actor does keyword/niche creator search, its payload, and per-run cost. Feasibility already proven by hand; the spike exists so the real payload drives the schema.
- Keyword → creators in that niche.
- Benchmarking: followers, views, comments per creator, against the seller's own.
- Outlier detection: posts performing far above their profile's average.
- Traction tracking: store the metrics already in every Apify payload (views, likes, comments, shares, saves) on every scrape. Zero extra cost.
- **Premium gating**, enforced in code. Per-seat quotas. Result caching.
- **Persist results into the hook corpus from day one**, before generation exists. The corpus compounds; starting late costs months of data.

**Done when:** a paying seller searches a keyword and sees a ranked, benchmarked list of creators — and a repeat search costs nothing.

---

### P8 — Scripts + Captions `p8-intel-generation`

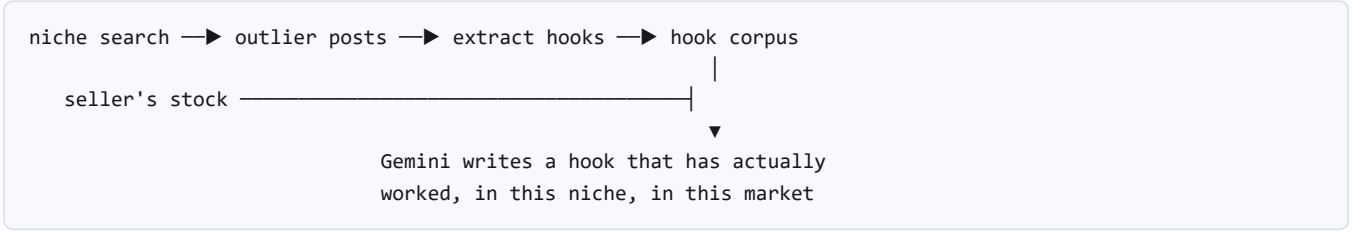
- **Hook corpus** — hooks extracted from P7's outliers, tagged by niche, language register (English / Swahili / Sheng), and the performance that earned their place.
- **Hook + caption generation** grounded in that corpus plus the seller's real stock. Not "write a viral hook", but "here are hooks that demonstrably worked in this niche in Kenya — write one for this product."
- **Script generation** built on the chosen hook.
- **Client reports** — per-seller performance over time.
- **Weekly digest** over WhatsApp.
- **Feedback loop** — record which generated hooks the seller filmed and how those videos performed. This signal is what makes the corpus better than a scrape.

**Done when:** a seller receives a digest naming a real trend, with a hook and script grounded in what is working in their niche.

---

## Where the moat is

Existing hook tools are not tailored to Kenya — hooks are culture- and language-bound, and English-market "viral formulas" do not transfer. That gap is the defensible position, and the architecture is a loop:




Every premium search feeds the corpus. The corpus improves generation. Better generation sells more seats. **A competitor can copy the feature but not the corpus.**

> **We are grounding, not training.** Fine-tuning means adjusting weights — expensive, slow, and needing far more data than we will have for a long time. Grounding retrieves real high-performing hooks into the prompt as examples: a fraction of the cost, and it improves the moment a new hook lands, with no retraining cycle. Revisit fine-tuning only if grounding measurably stops being enough.
>
> **Where RAG might finally earn its place.** Fetching hooks *by niche tag* is a plain lookup. But "find hooks similar in spirit to this product" is a genuine similarity problem — the first in the project. Recognise it if it arrives; don't reach for it early.

---

# Phase 4

### P9 — Hardening + pilot `p9-pilot`

- Rate limits, Gemini video cost caps, secrets audit, incident-grade logs.
- Onboard 2–3 real Kenyan sellers.
- One real order, end to end.

**Done when:** money moves for real, unattended.

---

## Cost rails

- **Apify, per-seller scraping** — at most once per day per seller, cached.
- **Apify, niche search (P7)** — the only cost a user can trigger repeatedly at will. Three guards, all required: **premium tier only** (cost tracks revenue), **per-seat quotas** (a paid seat is not unlimited), **result caching** (never pay twice for a keyword).
- **Gemini video** — the priciest tier of the cascade. Once ever per video id. TIKTOK hit the free-tier cap (20/day) on 2026-07-25; billing stays enabled.
- **Drafting is on-demand**, split out of ingest so ingest stays cheap.

> Gating niche search behind premium makes the most expensive feature the one that proves willingness to pay. Commerce acquires sellers free; Intel is the upgrade.

---

## Deliberately not in scope

- **SokoLedger** — Phase 2. On-device M-Pesa SMS capture and the Kotlin agent. Order-level payment rails (P4) are not the same thing.
- **Generative virtual try-on** — Phase 2, feature-flagged, cost-capped.
- **Marketplace search across sellers** — needs seller density that does not exist.
- **Fine-tuning on Kenyan hooks** — ground first; revisit only if that stops working.
- **WhatsApp Flows** — the in-app browser makes it unnecessary. Revisit only if Flows would demonstrably beat the webview.

---

## Open questions

1. **Does Aug 19 still mean anything?** That date came from the 14-day plan. P2 depends on Meta's approval clock, which is outside our control.
2. **Premium pricing** — the original tiers (Freemium / Pro KSh 499 / Growth KSh 999) predate Intel being this large. Intel is now the main upgrade reason.
3. **Seller surface** — sellers manage stock and review drafts. In the WhatsApp chat, in the same webview, or both?

### Answered

- ~~**WhatsApp Flows vs webview**~~ — **webview**, via the WhatsApp in-app browser (2026-08-05). Removes the Flows approval dependency entirely.
- ~~**Web interface**~~ — it is the storefront. One server-rendered surface serving both the in-app browser and public discoverability.
- ~~**Niche search feasibility**~~ — confirmed by hand against Apify, 2026-08-05.

---